

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт
Завдання №1.1
з дисципліни
“Прикладне статистичне моделювання”

Виконав:

студент групи Пма-42
Думанський Юрій

Керівник:

проф. кафедри
моделювання соціально-
економічних процесів
Сеньо П. С.

Тема:

Моделювання та розігрування випадкових змінних.

Завдання:

1. Змоделювати дискретну випадкову змінну, розподілену за біномним законом.

Математичне сподівання $\frac{2(i+1)(5i+1)}{(2i+1)}$, дисперсія $\frac{2i(i+1)(5i+1)}{(2i+1)^2}$, при $i = 10$
(порядковий номер у списку групи)

2. Змоделювати випадкову змінну η , функція розподілу якої

$$F(x) = \begin{cases} 0, & \text{якщо } x \leq 1 \\ \frac{(x-1)^2}{4}, & \text{якщо } 1 < x \leq 3 \\ 1, & \text{якщо } x > 3 \end{cases}$$

Хід роботи:**Пункт 1.**

Обчислено дисперсію та мат сподівання згідно формул

$$E\eta = \frac{2(10+1)(5 \cdot 10 + 1)}{2 \cdot 10 + 1} = \frac{2 \cdot 11 \cdot 51}{21} = \frac{374}{7}$$

$$D\eta = \frac{2 \cdot 10(10+1)(5 \cdot 10 + 1)}{(2 \cdot 10 + 1)^2} = \frac{20 \cdot 11 \cdot 51}{21^2} = \frac{11220}{441} = \frac{3740}{147}$$

З формули $D\eta = E\eta \cdot (1 - p)$ обчислено p :

$$1 - p = \frac{\frac{3740}{147}}{\frac{374}{7}} = \frac{3740}{147} \cdot \frac{7}{374}$$

$$1 - p = \frac{10}{21}$$

$$p = 1 - \frac{10}{21} = \frac{11}{21} \approx 0,524$$

Аналогічно з формули математичного сподівання $n\bar{p} = E\eta$ обчислено кількість випробувань n

$$n \cdot \frac{11}{21} = \frac{374}{7}$$

$$n = \frac{374}{7} \cdot \frac{21}{11}$$

$$n = 34 \cdot 3 = 102$$

Формула Бернуллі для обчислення ймовірності випадіння кожного значення

$$p_k = \binom{n}{k} p^k (1-p)^{n-k}$$

Алгоритм моделювання дискретної випадкової змінної:

Відрізок $[0,1]$ на числовій осі розбивається на n частин (інтервалів) за допомогою точок $p_1, p_1 + p_2, \dots, p_1 + p_2 + \dots + p_n = 1$

Утворені інтервали мають такий вигляд: $[0, p_1], (p_1, p_1 + p_2]$, і так далі аж до $(p_1 + p_2 + \dots + p_{n-1}, 1]$. Цим інтервалам присвоюються порядкові номери $1, 2, 3, \dots, n$.

За допомогою датчика випадкових чисел генерується випадкове число $\gamma_1 \in (0,1)$. Далі визначається, у який саме з побудованих інтервалів потрапило число γ_1 . Якщо число γ_1 потрапило в інтервал з номером i , це означає, що випадкова змінна η набула значення x_i . Для отримання вибірки потрібного обсягу N процес генерації γ та пошуку інтервалу повторюється необхідну кількість разів.

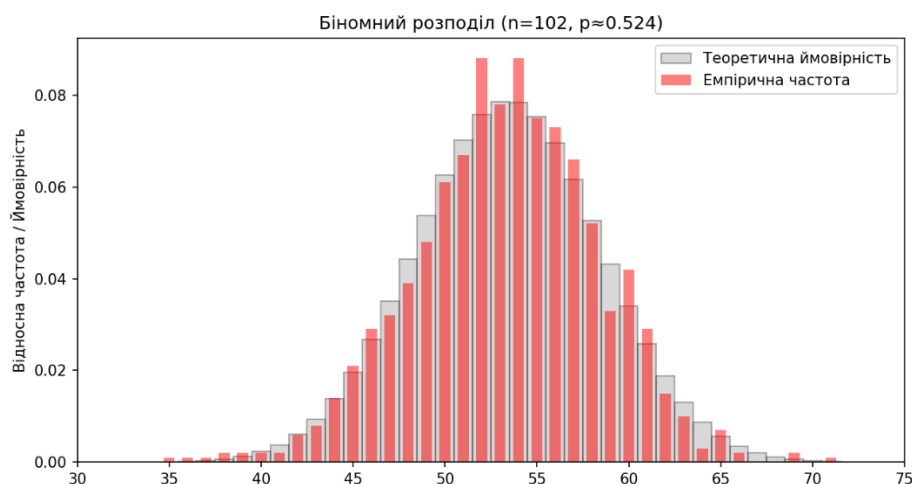


Рис. 1. Змодельована дискретна змінна при $N=1000$

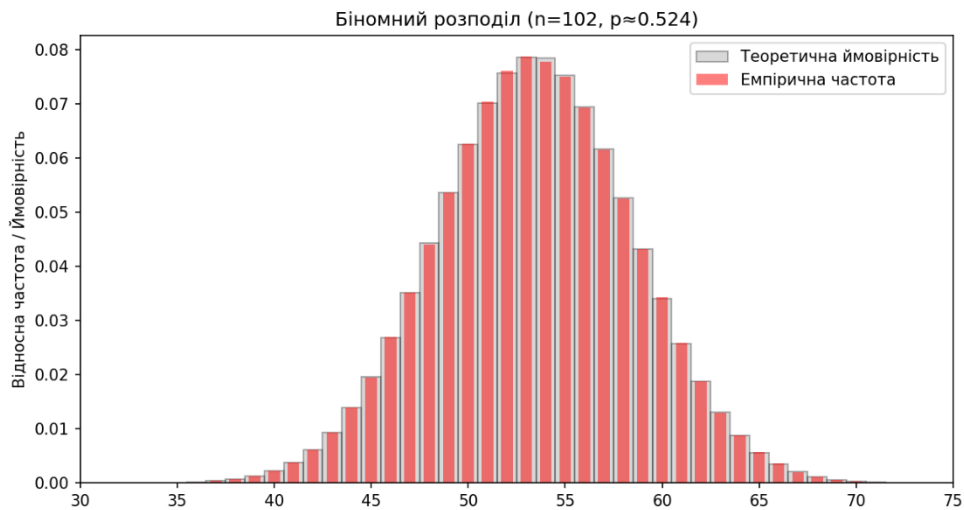


Рис. 2. Змодельована дискретна змінна при $N=1\,000\,000$

Пункт 2.

Для неперервної випадкової змінної алгоритм схожий.

Спочатку з датчика відчитується одне випадкове число $\gamma_1 \in (0,1)$.

Після цього розв'язується функціональне рівняння відносно x :

$$\int_{-\infty}^x q(t)dt = \gamma_1$$

Корінь цього рівняння x_1 є першим значенням випадкової змінної η . Для генерації наступних N значень x ці кроки повторюються.

Для функції розподілу із завдання, $\frac{(x-1)^2}{4}$, напроміжку $[1; 3]$, прирівнюємо її до згенерованого числа γ :

$$\frac{(x-1)^2}{4} = \gamma$$

Виражаємо x через γ :

$$(x-1)^2 = 4\gamma$$

$$x-1 = 2\sqrt{\gamma}$$

$$x = 1 + 2\sqrt{\gamma}$$

Після генерування і підстановки числа γ , змодельована випадкова змінна набуває значення x .

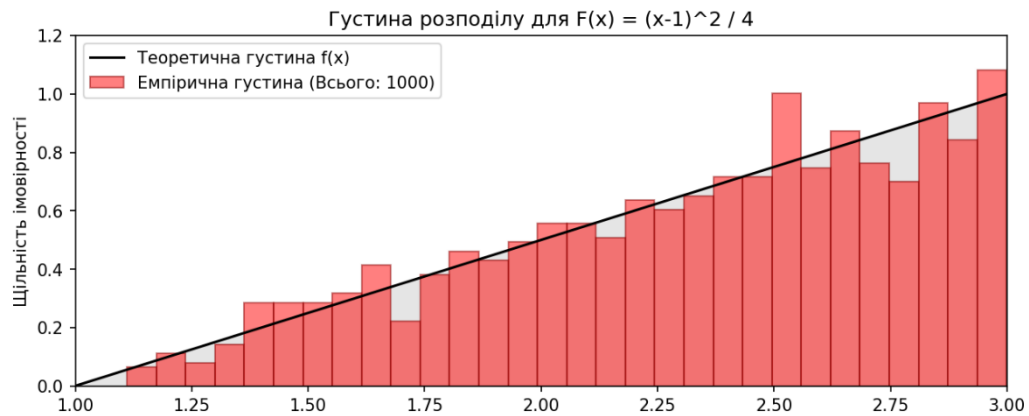


Рис. 3. Змодельована неперервна змінна при $N=1000$

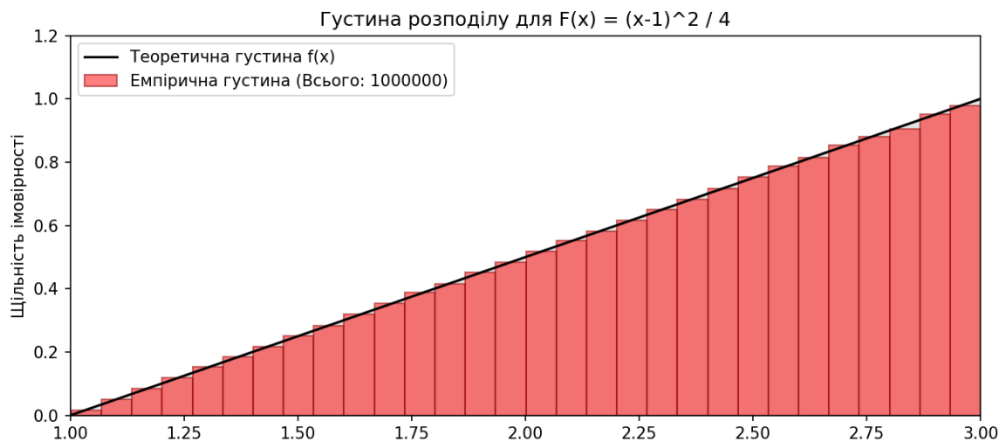


Рис. 2. Змодельована неперервна змінна при $N=1\,000\,000$

Програми написані на мові програмування Python, з використанням бібліотек `numpy` та `matplotlib` для математичних обчислень та графічного інтерфейсу. Код наведений в додатку 1.

Висновки:

У ході виконання лабораторної роботи було успішно реалізовано алгоритм моделювання дискретних випадкових змінних. Візуальне порівняння теоретичної та емпіричної гістограм підтвердило коректність розрахунків та коректність роботи алгоритму.

Додаток 1. Код програми.

Програма для пункту 1.

```
import matplotlib.pyplot as plt
from matplotlib.widgets import Button, TextBox
import random

def get_cumulative_sum(probabilities):
    cdf = []
    total = 0.0
    for p in probabilities:
        total += p
        cdf.append(total)
    return cdf

def get_frequencies(data_list):
    if not data_list:
        return [], []

    counts = {}
    for item in data_list:
        counts[item] = counts.get(item, 0) + 1

    unique_vals = list(counts.keys())
    unique_vals.sort()

    total_items = len(data_list)
    frequencies = [counts[val] / total_items for val in unique_vals]

    return unique_vals, frequencies

class VariableModel:
    """
    Stat modeling
    """
    def __init__(self, values, cdf):
        self.values = values
        self.cdf = cdf
    def search_sorted_index(cdf, gamma):
        for i, val in enumerate(cdf):
            if gamma <= val:
                return i
        return len(cdf) - 1
    def sample(self, num_points):
        gammas = [random.random() for _ in range(num_points)]
        sampled_results = []

        for g in gammas:
            index = self.search_sorted_index(self.cdf, g)
            sampled_results.append(self.values[index])
        return sampled_results, gammas

class BinomialModel:
    """Generating a Binomial Distribution."""
    def __init__(self, n, p):
        self.n = n
        self.p = p
        self.k_values = []
        self.pmf = []
        self.cdf = []
        self._calculate()

    def _calculate(self):
        self.k_values = list(range(self.n + 1))
        self.pmf = [(1 - self.p)**self.n

        for k in range(1, self.n + 1):
```

```

        next_p = self.pmf[-1] * ((self.n - k + 1) / k) * (self.p / (1 -
self.p))
        self.pmf.append(next_p)
        self.cdf = get_cumulative_sum(self.pmf)
class DistributionSimulatorUI:
    def __init__(self):
        # Setup Models
        self.binomial_model = BinomialModel(n=102, p=11/21)
        self.sampler = VariableModel(
            values=self.binomial_model.k_values,
            cdf=self.binomial_model.cdf
        )
        # State
        self.num_points = 1
        self.history_k = []
        self.history_gamma = []

        # Initialize UI
        self.setup_ui()

    def setup_ui(self):
        self.fig, (self.ax_dist, self.ax_line) = plt.subplots(
            2, 1, figsize=(10, 8), gridspec_kw={'height_ratios': [60, 1]}
        )
        self.ax_line.set_visible(False)
        plt.subplots_adjust(bottom=0.3, hspace=0.3)

        # Text Box
        ax_text = plt.axes([0.45, 0.15, 0.15, 0.05])
        self.textbox = TextBox(ax_text, 'К-сть точок (N): ', initial='1')
        self.textbox.on_submit(self.update_count)

        # Button
        ax_btn = plt.axes([0.45, 0.05, 0.2, 0.08])
        self.btn = Button(ax_btn, 'Згенерувати')
        self.btn.on_clicked(self.on_generate_clicked)

        self.draw_theoretical()
        plt.show()

    def update_count(self, text):
        try:
            val = int(text)
            self.num_points = max(1, val)
        except ValueError:
            self.textbox.set_val(str(self.num_points))

    def draw_theoretical(self):
        self.ax_dist.clear()
        self.ax_line.clear()

        # Fetch data from our model
        k_arr = self.binomial_model.k_values
        pmf = self.binomial_model.pmf
        cdf = self.binomial_model.cdf

        title = f'Біномний розподіл (n={self.binomial_model.n},
p≈{self.binomial_model.p:.3f}) '

        self.ax_dist.set_xlim(30, 75)
        self.ax_line.axvspan(0, 1, alpha=0.1, color='blue')

        # Draw CDF boundaries on the line plot
        for p in cdf[35:70]:
            self.ax_line.axvline(p, color='white', linewidth=0.5)

```

```

        self.ax_dist.bar(k_arr, pmf, color='gray', alpha=0.3, width=1.0,
edgecolor='black', label='Теоретична ймовірність')
        self.ax_dist.set_title(title)
        self.ax_dist.set_ylabel('Відносна частота / Ймовірність')
        self.ax_dist.legend(loc='upper right')

        self.ax_line.set_xlim(0, 1)
        self.ax_line.set_ylim(0, 1)
        self.ax_line.get_yaxis().set_visible(False)
        self.ax_line.set_title("Відрізок [0, 1] (Інтервали ймовірностей)")

def on_generate_clicked(self, event):
    # Delegate math generation to the separated sampler
    new_k, new_gammas = self.sampler.sample(self.num_points)

    self.history_k.extend(new_k)
    self.history_gamma.extend(new_gammas)

    print(f"{len(self.history_k)} точок згенеровано.")
    print(f"{len(self.history_gamma)} γ згенеровано.")
    self.update_plots()

def update_plots(self):
    # Remove old empirical bars
    for patch in
reversed(self.ax_dist.patches[len(self.binomial_model.k_values):]):
        patch.remove()

    # Remove old scatter points on the line
    [line.remove() for line in self.ax_line.lines[1:]]

    # Calculate empirical frequencies using our custom math utility
    unique_k, frequencies = get_frequencies(self.history_k)

    # Draw updated empirical data
    label = 'Емпірична частота' if len(self.history_k) == self.num_points
else ""
        self.ax_dist.bar(unique_k, frequencies, color='red', alpha=0.5,
width=0.6, label=label)

    # Jitter y-values for the line plot (using standard random)
    recent_gammas = self.history_gamma[-self.num_points:]
    y_jitter = [random.uniform(0.2, 0.8) for _ in range(len(recent_gammas))]
    self.ax_line.plot(recent_gammas, y_jitter, 'ko', markersize=3,
alpha=0.5)

    # Fix legend duplicates
    handles, labels = self.ax_dist.get_legend_handles_labels()
    by_label = dict(zip(labels, handles))
    self.ax_dist.legend(by_label.values(), by_label.keys(), loc='upper
right')

    self.fig.canvas.draw_idle()

if __name__ == '__main__':
    app = DistributionSimulatorUI()

```

Програма для пункту 2.

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Button, RadioButtons, TextBox

class ContinuousSimulator:

```



```

def __init__(self):
    self.dists = {
        'F(x) = (x-1)^2 / 4': {
            'x_min': 1.0,                # Нижня межа інтервалу
            'x_max': 3.0,                # Верхня межа інтервалу
            'pdf': lambda x: (x - 1) / 2,    # Похідна від F(x)
            'cdf': lambda x: ((x - 1)**2) / 4, # Функція розподілу
            'inv_cdf': lambda g: 1 + 2 * np.sqrt(g) # Обернена функція
        }
    }
    self.current_name = 'F(x) = (x-1)^2 / 4'
    self.dist = self.dists[self.current_name]
    self.num_points = 1

    self.gen_x = []
    self.gen_gamma = []
    self.setup_ui()

def setup_ui(self):
    self.fig, (self.ax_pdf, self.ax_cdf) = plt.subplots(2, 1, figsize=(10, 9),
    gridspec_kw={'height_ratios': [200, 1.5]})
    self.ax_cdf.set_visible(False)
    plt.subplots_adjust(bottom=0.25, hspace=0.35)
    # ax_radio = plt.axes([0.1, 0.05, 0.25, 0.12])
    # self.radio = RadioButtons(ax_radio, list(self.dists.keys()))
    # self.radio.on_clicked(self.change_dist)

    ax_text = plt.axes([0.45, 0.12, 0.15, 0.05])
    self.textbox = TextBox(ax_text, 'К-сть точок (N): ', initial='1')
    self.textbox.on_submit(self.update_count)

    ax_btn = plt.axes([0.45, 0.05, 0.2, 0.06])
    self.btn = Button(ax_btn, 'Згенерувати')
    self.btn.on_clicked(self.generate)

    self.draw_theoretical()
    plt.show()

def update_count(self, text):
    try:
        val = int(text)
        self.num_points = max(1, val)
    except ValueError:
        self.textbox.set_val(str(self.num_points))

def change_dist(self, label):
    self.current_name = label
    self.dist = self.dists[label]
    self.gen_x.clear()
    self.gen_gamma.clear()
    self.draw_theoretical()
    self.fig.canvas.draw_idle()

def draw_theoretical(self):

```

```

self.ax_pdf.clear()
self.ax_cdf.clear()

x_vals = np.linspace(self.dist['x_min'], self.dist['x_max'], 500)
pdf_vals = self.dist['pdf'](x_vals)
self.ax_pdf.plot(x_vals, pdf_vals, color='black', linewidth=1.5, label='Теоретична
густина f(x)')
self.ax_pdf.fill_between(x_vals, pdf_vals, color='gray', alpha=0.2)
self.ax_pdf.set_title(f"Густина розподілу для {self.current_name}")
self.ax_pdf.set_xlim(self.dist['x_min'], self.dist['x_max'])
self.ax_pdf.set_ylim(0, max(pdf_vals) * 1.2)
self.ax_pdf.set_ylabel('Щільність імовірності')
self.ax_pdf.legend(loc='upper left')

cdf_vals = self.dist['cdf'](x_vals)
self.ax_cdf.plot(x_vals, cdf_vals, color='blue', linewidth=2, label='Ф-я розподілу F(x)')
self.ax_cdf.set_title("Метод обернених функцій ( $\gamma \rightarrow x$ )")
self.ax_cdf.set_xlim(self.dist['x_min'], self.dist['x_max'])
self.ax_cdf.set_ylim(0, 1)
self.ax_cdf.set_xlabel('Згенероване значення (x)')
self.ax_cdf.set_ylabel('Випадкове число ( $\gamma$ )')
self.ax_cdf.legend(loc='upper left')

def generate(self, event):
    gammas = np.random.uniform(0, 1, self.num_points)

    xs = self.dist['inv_cdf'](gammas)

    self.gen_gamma.extend(gammas)
    self.gen_x.extend(xs)

    self.update_plots(new_gammas=gammas, new_xs=xs)

def update_plots(self, new_gammas, new_xs):
    self.draw_theoretical()

    self.ax_pdf.hist(self.gen_x, bins=30, density=True, color='red', alpha=0.5,
                     edgecolor='darkred', label=f'Емпірична густина (Всього:
{len(self.gen_x)})')
    self.ax_pdf.legend(loc='upper left')

    self.ax_cdf.scatter(self.gen_x, self.gen_gamma, color='black', s=15, zorder=5)

    if self.num_points <= 10:
        for g, x in zip(new_gammas, new_xs):
            self.ax_cdf.plot([self.dist['x_min'], x], [g, g], color='red', linestyle='--',
                             alpha=0.6)
            self.ax_cdf.plot([x, x], [0, g], color='red', linestyle='--', alpha=0.6)

    self.fig.canvas.draw_idle()

if __name__ == '__main__':
    app = ContinuousSimulator()

```